



SESSION 5

MORE BUILT-IN AND LOCAL MODULES

Learning Objectives

In this session, students will learn to:

- Explain the purpose and usage of `querystring` in Node.js
- Describe how to send emails using Node.js
- Describe the process of uploading files using Node.js
- Explain the concept of local Modules and their application in Node.js

Node.js includes several built-in and third-party modules, each tailored to simplify specific tasks. For example, the `querystring` module simplifies the parsing and manipulation of query strings within Universal Resource Locators (URLs), making data management in Web applications more straightforward.

The `nodemailer` module allows users to send emails programmatically in Node.js, enhancing communication within Node.js applications. Node.js also offers tools to simplify the process of uploading files, a common requirement in Web applications.

Local modules enable users to create custom code modules for better organization and reusability in Web projects.

5.1 `querystring` Module

Node.js provides the `querystring` module which is used to format and parse URLs. It is a valuable tool for passing data from one page to another in Web applications.

In simpler terms, this module simplifies working with URLs, allowing the user to add extra data in the form of key-value pairs to a URL. These pairs correspond to the specific information that is passed to a Web Server as part of an HTTP request. The information is embedded in the URL, typically after the `?`.

Table 5.1 shows two examples of query strings.

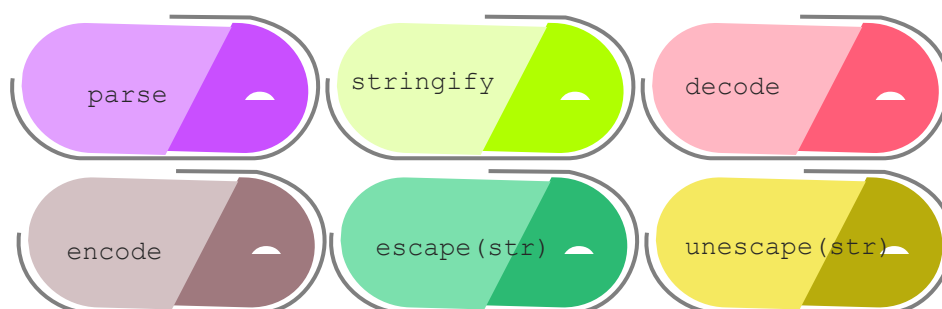
URL	Query String
https://exampleofquery.com/string/is?example=Query	example=Query
https://exampleofquery.com/path/of/the/page/is?example=Query&partof=module	example=Query&partof=module

Table 5.1: Sample Query Strings

5.1.1 `querystring` Methods

Node.js offers a valuable resource in the form of the `querystring` module, which extends beyond URL parsing.

Some of the essential methods in the `querystring` module are as follows:



These methods are valuable to Web developers, providing tools to efficiently manage data embedded within URLs.

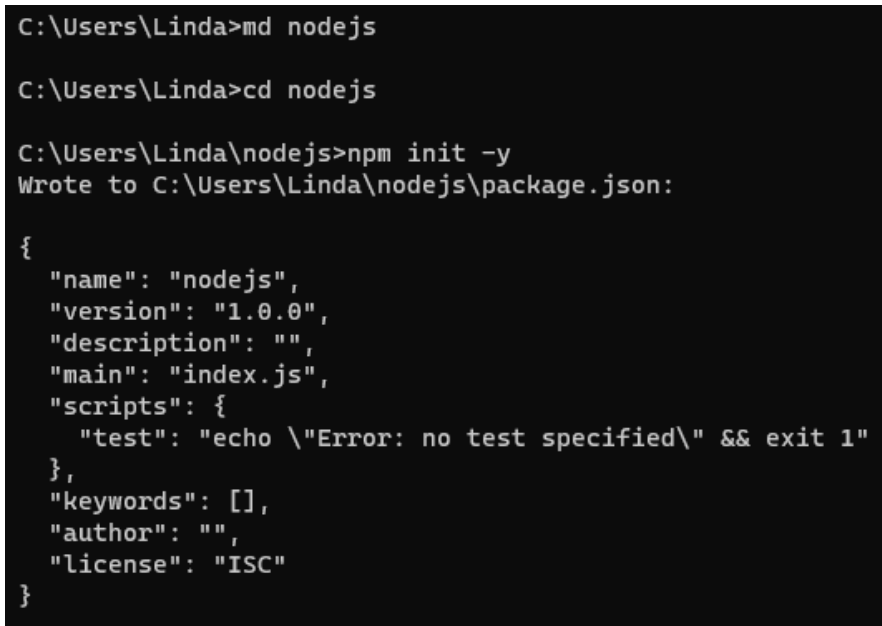
The first step to explore the use of each of these methods is to set up a basic Node.js application.

To set up a basic Node.js application:

1. Open the command prompt and navigate to a folder to create the Node.js project. If an existing Node.js project folder does not exist, create one.
2. To initiate a new Node.js project, run the command:

```
npm init -y
```

The `-y` flag is used to skip manual entry of project details. The command executes as shown in Figure 5.1.



```
C:\Users\Linda>md nodejs  
  
C:\Users\Linda>cd nodejs  
  
C:\Users\Linda\nodejs>npm init -y  
Wrote to C:\Users\Linda\nodejs\package.json:  
  
{  
  "name": "nodejs",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

Figure 5.1: Basic package.json File

The command generates a basic `package.json` file, serving as the foundation of any Node.js project.

3. Within the project folder, create a new JavaScript file named `querystringexample.js`. The first line of the `querystringexample.js` file should include the import statement for the `querystring` module, as shown in Figure 5.2.

```
const querystring = require('querystring');
```

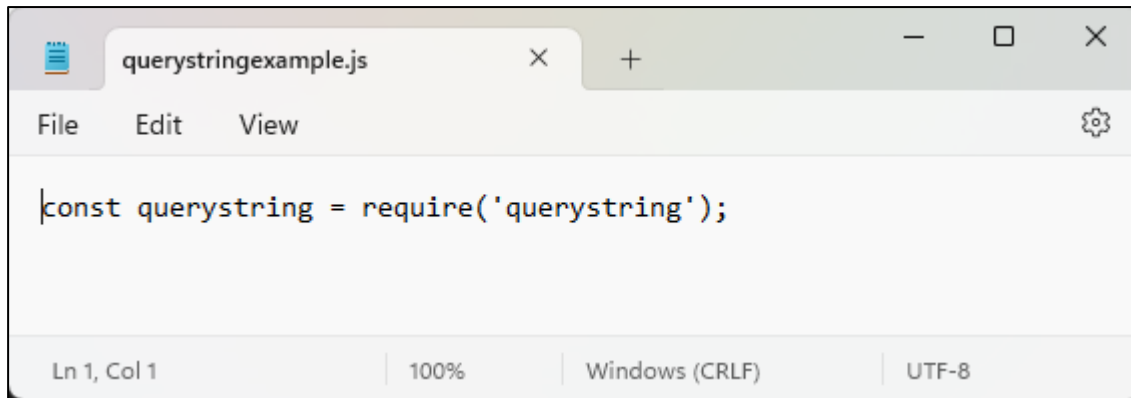


Figure 5.2: querystringexample.js File

4. Save the file.

The Node.js project is successfully set up. Now, the methods of the `querystring` module can be used.

5.1.2 `querystring.parse` Method

The `querystring.parse` method is utilized for parsing the URL query string into an object that contains key-value pairs. It is important to note that the resulting object is not a JavaScript object. Therefore, standard object methods such as `obj.toString` or `obj.hasOwnProperty` cannot be used with the object returned by the `parse` method.

By default, the method assumes the latest UTF-8 encoding format unless an alternative encoding format is specified. Only if the project has specific requirements for an alternative character encoding should the `decodeURIComponent` option be considered. However, it is advisable to stick with UTF-8 encoding, which is the standard and encompasses all international characters.

The syntax for the `querystring.parse` method is:

```
querystring.parse( str[, sep[, eq[, options]]]) )
```

The `parse` method accepts four parameters:

str (Required)

This parameter specifies the query string that has to be parsed.

Sep (Optional)

This parameter specifies the substring that delimits the key-value pairs in the query string. The default and commonly used value is `&`.

Eq (Optional)

This parameter specifies the substring used to separate keys and values in the query string. The default and commonly used value is `=`.

Options (Optional)

This parameter specifies the object field used to modify the method's behaviour. It includes:

decodeURIComponent: A function that specifies the encoding format within the query string. The default value is `querystring.unescape()`.

maxKeys: A number that determines the maximum number of keys to be parsed. A value of 0 removes all counting limits, allowing the parsing of any number of keys. The default value is set at 1000.

The six options that can be used in the `querystring.parse` method are as follows:

- **Parsing a URL query string**

The `querystring.parse` method is used to parse a query string into a JavaScript object. Code Snippet 1 lists the code to do this.

Code Snippet 1:

```
const querystring = require("querystring");

let queryStringURL = "name=John&age=30&city=New+York";
let parsingObj = querystring.parse(queryStringURL);
console.log("Parsed Query 1:", parsingObj);
```

In Code Snippet 1, the `queryStringURL` variable contains the key-value pairs separated by the `&` symbol. The result is stored in the `parsingObj` variable. To try this example:

1. Open VS Code.
2. Type the code given in Code Snippet 1.
3. Save the file as `querystringexample.js`.
4. Open Command Prompt and run the command as:

```
node querystringexample.js
```

The parsed object is printed on the console as shown in Figure 5.3.

```
C:\Users\Linda\nodejs>node querystringexample.js
Parsed Query 1: [Object: null prototype] { name: 'John', age: '30', city: 'New York' }
```

Figure 5.3: Parsed Object

- **Stringifying an object into a URL query string**

The `querystring.stringify` method is used to convert an object into a URL query string. Code Snippet 2 lists the code to do this.

Code Snippet 2:

```
const querystring = require("querystring");
let dataObj = { name: "Alice", age: 25, city: "Los Angeles" };
let queryString = querystring.stringify(dataObj);
console.log("\nStringified Query 2:", queryString);
```

In Code Snippet 2, a JavaScript object `dataObj` has the key-value pairs. The result is stored in the `queryString` variable and printed on the console as shown in Figure 5.4.

```
C:\Users\Linda\nodejs>node objtoqs.js  
Stringified Query 2: name=Alice&age=25&city=Los%20Angeles
```

Figure 5.4: Stringed Query

- **Parsing a query string with custom delimiters**

The `querystring.parse` method can be used to parse custom query strings. Code Snippet 3 lists the code to do this.

Code Snippet 3:

```
const querystring = require("querystring");  
  
queryStringURL = "itemA|1&itemB|2&itemC|3";  
parsingObj = querystring.parse(queryStringURL, "&", "|");  
console.log("\nParsed Query 3:", parsingObj);
```

In Code Snippet 3, a custom-formatted query string has the `&` character as the separator between key-value pairs and the `|` character as the delimiter between keys and values. This custom string is stored in the `queryStringURL` variable. The `querystring.parse` method uses the separator and custom delimiter to parse the given query string. The parsed data is stored in the `parsingObj` variable as an object. The result is printed on the console as shown in Figure 5.5.

```
C:\Users\Linda\nodejs>node customqs.js  
Parsed Query 3: [Object: null prototype] { itemA: '1', itemB: '2', itemC: '3' }
```

Figure 5.5: Parsed Object with Custom Delimiters

- **Parsing a query string with `maxKeys` set to 1**

The `maxKeys` option limits the number of keys parsed from the query string. Code Snippet 4 lists the code to demonstrate this.

Code Snippet 4:

```
const querystring = require("querystring");
queryStringURL =
"fruit=apple&fruit=banana&fruit=cherry&fruit=orange";
parsingObj = querystring.parse(queryStringURL, "&", "=", {
maxKeys: 1 });
console.log("\nParsed Query 4:", parsingObj);
```

In Code Snippet 4, the variable `queryStringURL` contains a query string with multiple key-value pairs. The `&` character separates pairs and the `=` character separates keys and values. Additionally, the `maxKeys` option is set to 1. Thus, when this code is executed, only one key-value pair will be parsed and the rest will be ignored. The result is stored in the `parsingObj` variable and printed on the console as shown in Figure 5.6.

```
C:\Users\Linda\nodejs>node maxkeys.js

Parsed Query 4: [Object: null prototype] { fruit: 'apple' }
```

Figure 5.6: Parsed Object with `maxKeys` Set to 1

Similarly, in Code Snippet 5, the `maxkeys` option is used to set the limit of key-value pairs to 2.

Code Snippet 5:

```
const querystring = require("querystring");
queryStringURL =
"fruit=apple&fruit=banana&fruit=cherry&fruit=orange";
parsingObj = querystring.parse(queryStringURL, "&", "=", {
maxKeys: 2 });
console.log("\nParsed Query 5:", parsingObj);
```

The result is printed on the console as shown in Figure 5.7.

```
C:\Users\Linda\nodejs>node maxkeys2.js

Parsed Query 5: [Object: null prototype] { fruit: [ 'apple', 'banana' ] }
```

Figure 5.7: Parsed Object with `maxKeys` Set to 2

If the `maxKeys` option is set to 0, then all the key-value pairs will be parsed. Code Snippet 6 lists the code to demonstrate this.

Code Snippet 6:

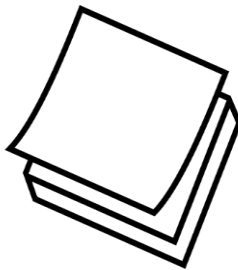
```
const querystring = require("querystring");
queryStringURL =
"fruit=apple&fruit=banana&fruit=cherry&fruit=orange";
parsingObj = querystring.parse(queryStringURL, "&", "=", {
maxKeys: 0 });
console.log("\nParsed Query 6:", parsingObj);
```

The result is printed on the console, as shown in Figure 5.8.

```
C:\Users\Linda\nodejs>node maxkeys0.js

Parsed Query 6: [Object: null prototype] {
  fruit: [ 'apple', 'banana', 'cherry', 'orange' ]
}
```

Figure 5.8: Parsed Object with `maxKeys` Set to 0



The `querystring.unescape` method is involved in decoding key-value pairs in a query string. In practice, the `querystring.unescape` is not directly used in the code. The `querystring.parse` method handles this decoding process, making it unnecessary to use `querystring.unescape` separately.

5.1.3 `querystring.stringify` Method

The `querystring.stringify` method is used to produce a query string from a given object that contains the key value pairs. This method works exactly opposite to the `querystring.parse` method.

The resultant query string may contain strings, numbers, and/or Boolean values. The process of converting an object to a query string is termed serialization of the object.

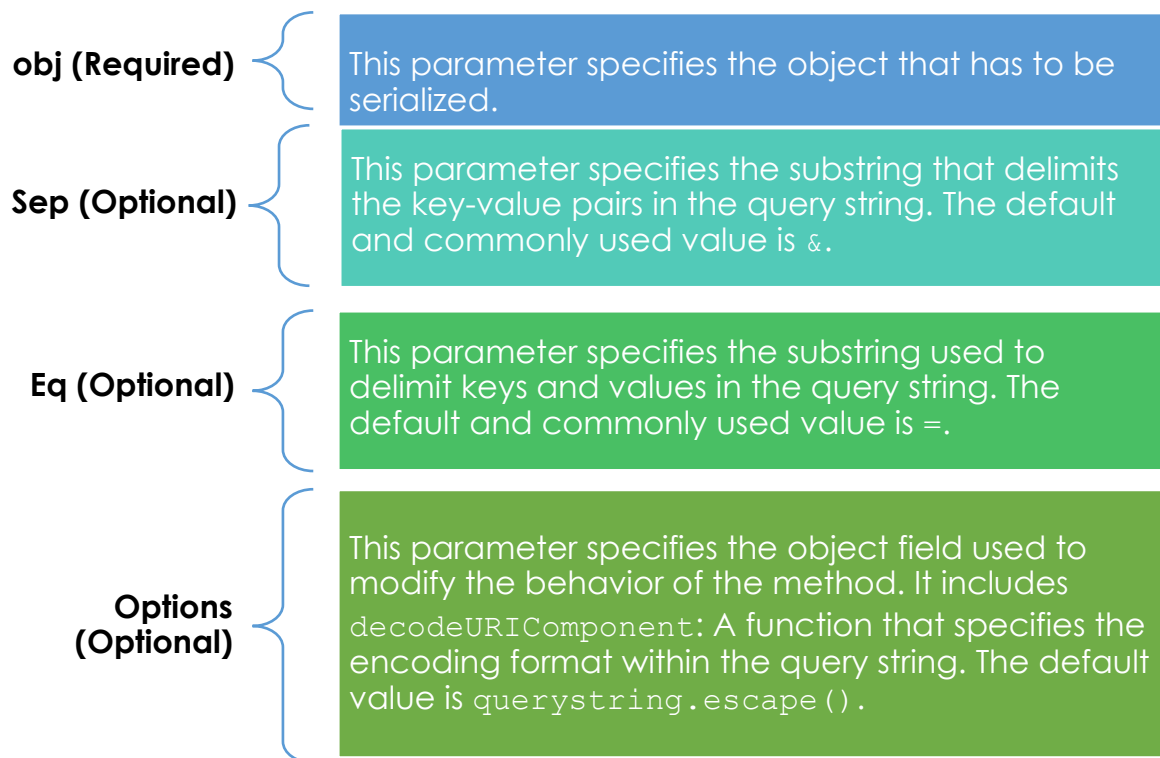
By default, the method assumes the use of the UTF-8 encoding format unless an alternative encoding format is specified. However, adhering to UTF-8 encoding is advisable as it is the standard encoding format encompassing all

international characters. In cases requiring alternative character encoding, the `decodeURIComponent` option should be used.

The syntax of the `querystring.stringify` method is:

```
querystring.stringify( obj[, sep[, eq[, options]]] )
```

The `querystring.stringify` method accepts four parameters:



The `querystring` module in Node.js allows the users to convert JavaScript objects into URL query strings in different ways.

Code Snippet 7 lists the code to demonstrate the default behavior of the `stringify` method when converting objects into URL strings. The code defines a user object, `user1` and initializes it with distinct properties such as `username`, `activity status`, and `roles`.

Code Snippet 7:

```
const querystring = require("querystring");

let user1 = {
  username: "Alice",
  active: true,
  roles: ["developer", "designer", "manager"],
};

let queryString1 = querystring.stringify(user1);
console.log("Query String 1:", queryString1);
```

The result serializes the `user1` object into a query string, using `&` as the separator and `=` to separate key-value pairs. The resulting query string is printed to the console as shown in Figure 5.9.

```
C:\Users\Linda\nodejs>node stringify.js
Query String 1: username=Alice&active=true&roles=developer&roles=designer&roles=manager
```

Figure 5.9: Stringed Query

Code Snippet 8 uses custom separators and delimiters to achieve object serialization. This showcases the flexibility of the `querystring` module in accommodating different formatting requirements.

Code Snippet 8:

```
const querystring = require("querystring");

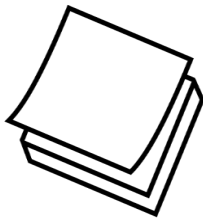
let user1 = {
  username: "Alice",
  active: true,
  roles: ["developer", "designer", "manager"],
};

let queryString1 = querystring.stringify(user1, "&", "&", ":");
console.log("Query String 1:", queryString1);
};
```

The result serializes the `user1` object into a query string, using a comma followed by a space (,) as the separator and a colon : to separate key-value pairs. The resulting query string is printed to the console as shown in Figure 5.10.

```
C:\Users\Linda\nodejs>node stringifycus.js
Query String 1: username:Alice, active:true, roles:developer, roles:designer, roles:manager
```

Figure 5.10: Stringed Query With Custom Separators



The `querystring.escape` method is used to encode key-value pairs in a query string. In practice, the `querystring.escape` method is not used directly in the code. The `querystring.stringify` method takes care of this encoding process, making it unnecessary to use `querystring.escape` separately.

5.1.4 `querystring.decode` Method

The `querystring.decode` method is nothing, but an alias for the `querystring.parse` method.

The `decode` method takes an input and processes it. The result is a JavaScript object, where each key becomes a property. The associated values can be single values or arrays when there are multiple values for the same key.

Code Snippet 9 shows the code to decode an URL into JavaScript object using the `decode` method.

Code Snippet 9:

```
// Import the querystring module
const querystring = require("querystring");

// Define a custom URL query string to be parsed
let customQueryString =
"product=smartphone&brand=Samsung&brand=Apple&available=true
";

// Use the decode() method on the string
let parsedData = querystring.decode(customQueryString);

// Print the parsed object to the console
console.log("Parsed Data:", parsedData);
```

In Code Snippet 9,

- The `querystring` module is imported to access its functionality.
- A custom URL query string variable, `customQueryString`, is defined and initialized with a value. This query string value is made up of key-value pairs separated by `&`.
- The `decode` method is used to parse the data.
- The parsed object is displayed on the console.

The processed data is printed to the console as shown in Figure 5.11.

```
C:\Users\Linda\nodejs>node decode.js
Parsed Data: [Object: null prototype] {
  product: 'smartphone',
  brand: [ 'Samsung', 'Apple' ],
  available: 'true'
}
```

Figure 5.11: Decoded Data

5.1.5 `querystring.encode` Method

The `querystring.encode` method is nothing, but an alias for `querystring.stringify` method.

The `querystring.encode` method is used to serialize an object into a URL query string. This method converts the properties of an object into key-value pairs, using the default separators `&` for pairs and `=` for separating keys and values.

Code Snippet 10 shows the code to encode a JavaScript object using the `encode` method.

Code Snippet 10:

```
const querystring = require("querystring");

let userData = {
  username: "Alice",
  active: true,
  roles: ["developer", "designer", "manager"],
};

let queryString1 = querystring.encode(userData);
console.log("Serialized Data 1:", queryString1);
```

In Code Snippet 10,

- The `querystring` module is imported to access its functionality.
- A JavaScript object, `userData`, is defined and initialized with values.
- The `encode` method is used to string the data.
- The stringed query is displayed on the console.

The stringed query is displayed on the console as shown in Figure 5.12.

```
C:\Users\Linda\nodejs>node encode.js  
Serialized Data 1: username=Alice&active=true&roles=developer&roles=designer&roles=manager
```

Figure 5.12: Serialized Data

5.2 nodemailer Module

Node.js itself does not have a built-in email module. If users want to work with email in Node.js, they must typically use a third-party library such as `nodemailer`.

`nodemailer` is a widely used library that simplifies the process of sending emails from Node.js applications. To install `nodemailer`, perform these steps:

1. Open a Command Prompt window.
2. To install `nodemailer`, at the Command Prompt, run the command as:

```
npm install nodemailer
```

This command downloads and installs the `nodemailer` module on the local system as shown in Figure 5.12.

```
C:\NodeJS Programs>npm install nodemailer  
added 1 package in 986ms
```

Figure 5.12: Installing nodemailer

After `nodemailer` is successfully installed, users can use it in their Node.js application.

To create an application to send an email from a Node.js application, perform these steps:

1. Open a VS Code file.
2. Import the `nodemailer` module, using the code as:

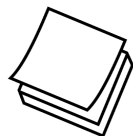
```
var nodemailer = require('nodemailer');
```

3. To set up Gmail as the email transport, in the same file, type the code as given in Code Snippet 11.

Code Snippet 11:

```
var emailClient = nodemailer.createTransport({  
  service: 'gmail',  
  auth: {  
    user: 'your_email@gmail.com',  
    pass: 'your_password'  
  }  
});
```

In Code Snippet 11, the `nodemailer.createTransport` function sets up the email transport. Here, the Gmail service is configured as email transport. To configure this transport, the user must provide the Gmail email address and password for authentication.



Specifying the password directly in the code is not recommended. Instead, it is better to use application-specific passwords or OAuth tokens for enhanced security.

3. To specify the contents of the email, in the same file, type the code as given in Code Snippet 12.

Code Snippet 12:

```
var emailContent = {  
  from: 'sender@gmail.com',  
  to: 'recipient@example.com',  
  subject: 'Hello from Node.js',  
  text: 'This is a test email from Node.js.'  
};
```

In Code Snippet 12, the `emailContent` object specifies email details, such as the sender's email address (`from`), the recipient's email address (`to`), the email subject (`subject`), and the email body text (`text`).

4. To send the email, in the same file, type the code as given in Code Snippet 13.

Code Snippet 13:

```
emailClient.sendMail(emailContent, function(error,
info){
    if (error) {
        console.log('Error: ' + error);
    } else {
        console.log('Email sent: ' + info.response);
    }
})
```

In Code Snippet 13, the `emailClient.sendMail` function sends the email. It requires the `emailContent` object as a parameter and a callback function to manage the result.

5. Save the file as `sendEmail.js`.
6. To run the application, open a Command Prompt window.
7. At the prompt, run the command as:

```
node senEmail.js
```

The application sends the email and displays an output as shown in Figure 5.13.

```
C:\NodeJS Programs>node sendEmail.js
Email sent: 250 2.0.0 OK 1700653891 q6-20020a17090a2e0600b00268b439a0cb5m1206910pjd.23 - gmail
```

Figure 5.13: Email Message Successfully Sent

If there is any issue, such as incorrect username and password provided to log on the email service, the application will display an error as shown in Figure 5.14.

```
C:\NodeJS Programs>node sendEmail.js
Error: Error: Invalid login: 535-5.7.8 Username and Password not accepted. Learn more at
535 5.7.8 https://support.google.com/mail/?p=BadCredentials w13-20020a1709027b8d00b001c9b35287aesm9637219pll.88 - gmail
```

Figure 5.14: Error in Sending Email Message

Users can also use `nodemailer` to send emails to a group of recipients using Node.js. To do this, the email addresses must be specified in the `to` property by separating each email address with commas, as shown in Code Snippet 14.

Code Snippet 14:

```
var emailDetails = {  
  from: 'your_email@gmail.com',  
  to: 'recipient1@example.com, recipient2@example.com',  
  subject: 'Hello from Node.js',  
  text: 'This is a test email from Node.js.'  
};
```

5.3 Uploading Files

Many Web applications allow users to upload files, such as images, documents, or other types of media. Node.js, however, lacks a built-in module dedicated to file upload processing. When it comes to managing file uploads in Node.js, users can use third-party modules, such as `formidable`.

The `formidable` module simplifies the process of receiving and processing uploaded files in Node.js applications. To use the `formidable` module in a Node.js application, perform these steps:

1. To install the `formidable` module, open a Command Prompt window.
2. At the Command Prompt, run the command as:

```
npm install formidable
```

This command downloads and installs the `formidable` module on the local system, as shown in Figure 5.15.

```
C:\NodeJS Programs>npm install formidable  
  
added 6 packages, and audited 8 packages in 2s  
  
1 package is looking for funding  
  run 'npm fund' for details  
  
found 0 vulnerabilities
```

Figure 5.15: Installing formidable Module

After `formidable` is successfully installed, users can use it in their Node.js application.

To create an application to allow users to upload files from a Node.js application, perform these steps:

1. To create an HTML form that the users can use to upload a file, open a file, and type the code given in Code Snippet 15.

Code Snippet 15:

```
const var_http = require('http');

var_http.createServer(function (req, res)
{
    res.writeHead(200, {'Content-Type': 'text/html'});

    res.write('<form action="submitFile" method="post"
    enctype="multipart/form-data">');

    res.write('<b>Select file to upload:</b><br><br>
    <input type="file" name="uploadfile"><br><br>');

    res.write('<input type="submit">');

    res.write('</form>');

    return res.end();
}).listen(3000, console.log("The HTTP Server is
listening on port 3000."));
```

The code in Code Snippet 15, creates an HTTP server using the `createServer` method. This server displays an HTML file that shows a label `Select a file to upload:`, with a **Choose File** button to choose the file to upload and a `Submit` button to upload the file. The elements of the form are created using the `write` method of the `res` object. The server is also configured to listen on port 3000 using the `listen` method of the HTTP server.

2. To configure the HTTP server to parse the uploaded file, in the same file, after the declaration of the `var_http` variable, type the code as:

```
const var_formidable = require('formidable');
```

This code creates an instance of the `formidable` module, which is required to handle the file upload.

To parse the uploaded file when the user clicks the **Submit** button on the form, in the same file, after the opening braces of `var_http.createServer(function (req, res,)` add the code given in Code Snippet 16.

Code Snippet 16:

```
if (req.url == '/submitFile')
{
    var form = new var_formidable.IncomingForm();
    form.parse(req, function (err, fields, files)
    {
        if (err)
        {
            console.error('No file selected',
                err);
            res.end('No file selected !!');
            return;
        }
        else
        {
            res.write('File Submitted');
            res.end();
        }
    });
}
```

The code in Code Snippet 16 uses the `parse` function of the `formidable` instance to parse the uploaded file. It also checks for any error when parsing the file, such as a file not uploaded. It displays the message `File Submitted` if the parsing is successful. If an error is encountered, It returns the message `No file selected !!`.

3. Move the code for displaying the upload file form into the `else` block as shown in Code Snippet 17.

Code Snippet 17:

```
else
{
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write('<form action="submitFile" method="post"
    enctype="multipart/form-data">');
    res.write('<b>Select file to upload:</b><br><br>
    <input type="file" name="uploadfile"><br><br>');
    res.write('<input type="submit">');
    res.write('</form>');
    return res.end();
}
}).listen(3000, console.log("The HTTP Server is
listening on port 3000."));
```

4. Save the file as `uploadFile.js`.
5. To run the application, open a Command Prompt window.
6. At the command prompt, run the command as:

```
node uploadFile.js
```

This command starts the HTTP server that listens on port 3000 and displays an output as shown in Figure 5.16.

```
C:\NodeJS Programs>node uploadFile.js
The HTTP Server is listening on port 3000.
```

Figure 5.16: Starting the HTTP Server

7. Open a Web browser window and navigate to the URL <http://localhost:3000>.

The browser window displays the HTML page with the form to upload the file as shown in Figure 5.17.

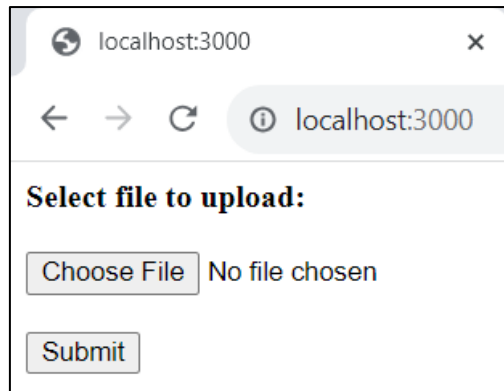


Figure 5.17: Form to Upload File

8. Click **Choose File** and from the local system, select a file to upload. The file name appears next to the **Choose File** button as shown in Figure 5.18.

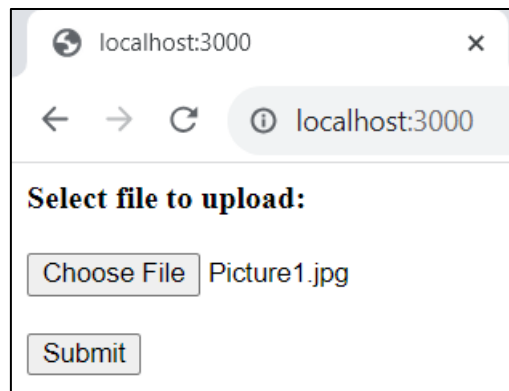


Figure 5.18: Selecting File to Upload

9. Click **Submit**. The file is uploaded and the message is displayed as shown in Figure 5.19.

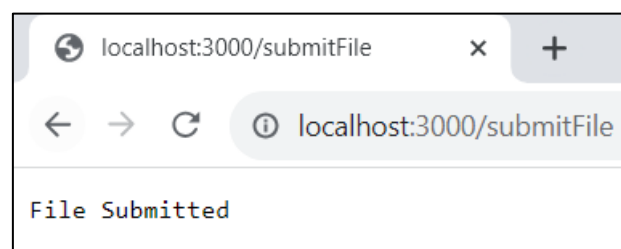


Figure 5.19: File Submitted

10. Open a new tab in the browser window and navigate to the URL <http://localhost:3000>.

The browser window displays the HTML page with the form to upload the file.

11. Click **Submit** without selecting any file.

The error message is displayed in the browser window as shown in Figure 5.20.

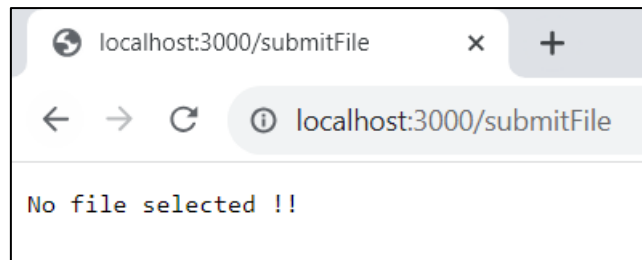


Figure 5.20: Error Message

5.4 Local Modules

Node.js developers often want to reuse pieces of code to save time. They achieve this by using local modules—custom-made modules created locally to fulfil specific requirements.

Local modules help in organizing the code into smaller, reusable parts, thereby simplifying the management and maintenance of the application. These modules also help implement encapsulation by hiding the code details and only exposing the functionality.

The local modules are created in a separate `.js` file and then, imported into the application, as required. These modules must be exported for them to be available for import into other applications. To export the modules, the `module.exports` function is used. This is a special function provided in Node.js, by default. This function is used to make a module available to other application. The application can import the exposed modules using the `require` method.

For example, consider the code in Code Snippet 18.

Code Snippet 18:

```
const MyFunc =
{
  msg: "This is a sample of using local modules.",

  addNum: function (a, b)
  {
    const add_result = a + b;
    console.log("The result is: ", a, "+", b, "=",
      add_result);
  },
  subNum: function (a, b)
  {
    if (a>=b)
    {
      const sub_result = a - b;
      console.log("The result is: ", a, "-", b, "=",
        sub_result);
    }
    else
    {
      const err = "Cannot subtract"
      console.log("The result is: ", a, "-", b, "=",
        err);
    }
  }
}

module.exports = MyFunc;
```

Code Snippet 18 creates a module named `MyFunc`. This module includes one variable `msg`, which stores a message. It also includes two functions `addNum` and `subNum`. The `addNum` function takes two parameters: `a` and `b`. It then, adds the parameters and stores the result in a variable named `add_result`. Finally, it displays a message. Similarly, the `subNum` function takes two parameters: `a` and `b`. It then, checks if `a` is larger than the `b`. If yes, it subtracts the second parameter from the first parameter and stores the result of the operation in a variable named `sub_result` and displays a message. If `a` is smaller than `b`, the message stored in the variable named `err` is displayed. Finally, the code uses the `module.exports` object to expose the `MyFunc` module.

This file is saved as `MyFunc.js`.

Next, consider the code in Code Snippet 19.

Code Snippet 19:

```
const localmod = require ("./MyFuncs.js");  
  
console.log(localmod.msg);  
  
localmod.addNum(10, 40);  
  
localmod.subNum(40, 10);  
  
localmod.subNum (10, 40);
```

Code Snippet 19 uses the `require` method to import the `MyFunc` module. It then calls the variables and functions included in the module.

This file is saved as `useLocalModule.js`.

To see the working of the local modules, open a Command Prompt window. Then, at the Command Prompt, run the command as:

```
node useLocalModule.js
```

The application executes and displays the output as shown in Figure 5.21.

```
C:\NodeJS Programs>node useLocalModule.js  
This is a sample of using local modules.  
The result is: 10 + 40 = 50  
The result is: 40 - 10 = 30  
The result is: 10 - 40 = Cannot subtract
```

Figure 5.21: Working of Local Modules

5.5 Summary

- The `querystring` module simplifies the parsing and manipulation of query strings within URLs.
- The essential methods of the `querystring` module are `parse`, `stringify`, `decode`, `encode`, `escape(str)`, and `unescape(str)`.
- Node.js relies on third-party libraries such as `nodemailer` for email-related tasks, allowing users to send emails programmatically and improve communication within applications.
- Node.js lacks a built-in module for file uploads but supports the usage of third-party modules such as `formidable`, which simplifies the process of receiving and handling uploaded files in Node.js applications.
- Local Modules enable users to create custom code modules for better organization and reusability in applications.
- The `module.exports` object is a fundamental part of Node.js, included in every JavaScript file by default.

Test Your Knowledge

1. Which of the following module does not come under Built-in modules?

- a) Request Module
- b) Email Module
- c) Local Module
- d) Export Module

2. What is the purpose of `Querystring` module in Built-in modules?

- a) Ability to send emails programmatically in Node.js
- b) Simplify the process of uploading files
- c) Create custom code modules
- d) Simplify the parsing and manipulation of query strings within URLs

3. What is the correct output of the given code:

```
const querystring = require("querystring");

let foodObj = { cuisine: "Italian", food: "Bruschetta",
city: "Italy" };
let queryString = querystring.stringify(foodObj);
console.log("\nStringified Query 2:", queryString);
```

- a) Stringified Query 2:
cuisine=Italian&food=Bruschetta&city=Italy
- b) Stringified Query:
cuisine=Italian&food=Bruschetta&city=Italy
- c) Error in the code
- d) Stringified Query 11:
cuisine=Italian&food=Bruschetta&city=Italy

4. Identify the missing statements in the given code in order to get this output:

Query String 1:
playerName=John&active=true&hobby=cricket&hobby=football

Code:

```
const querystring = require("querystring");  
let // Add missing code here  
};  
  
let queryString1 = querystring.stringify(player1);  
console.log("Query String 1:", queryString1);  
  
player2 = {  
  playerName: "Anna",  
  active: false,  
  hobby: ["golf", "ice hockey "],  
};
```

- a) player1 = {
 playerName: "John",
 active: true,
 hobby: ["cricket", "football"],
}
- b) player2 = {
 playerName: "Lee",
 active: true,
 hobby: ["cricket", "football"],
}
- c) player1 = {
 playerName: "John",
 active: false,
 hobby: ["cricket", "football"],
}
- d) player2 = {
 playerName: "Lee",
 active: false,
 hobby: ["cricket", "football"],
}

5. What is the use of `querystring.encode` method in Node.js?

- a) This method is used to decode the object
- b) This method is used to serialize an object into a URL query string
- c) This method is used in encoding key-value pairs
- d) None of these

Answers to Test Your Knowledge

1	a
2	d
3	a
4	a
5	b

Try It Yourself

1. You are working on a Node.js application. You want to parse and extract data from URL query string.

- a) Import the `querystring` module.
- b) Parse the query string using this data:
 1. `playerName: Jack`
 2. `city: Canada`
 3. `hobby: snowboarding`
- c) Display parsed data in the console.

2. Create a program to convert JavaScript objects into URL query strings.

- a) Define two `book` objects, `book1` and `book2`, each with distinct properties such as `bookName`, `activity status`, and `typeOfBook` using the given data:

```
book1
bookName= Captain Chuckle
activity status= false
typeOfBook= Drama
```

```
book2
bookName= Funny Bones
activity status= active
typeOfBook= Comedy
```

- b) To serialize the object, use `book2`.
- c) Use the `stringify` method on the object.
- d) Display the result on the console.

3. Write a Node.js application to encode the data:

```
playerName: Jack,  
city: Canada,  
hobby: snowboarding
```

Also add code to decode the data:

```
String_decode:  
playerName=Jack&city=Canada&hobby=snowboarding&available=  
true
```

- a) Import the `querystring` module.
 - b) Define a custom URL query string to be parsed.
 - c) Use the `decode` method on the `String_decode`.
 - d) Print the parsed object to the console.
 - e) Define a custom object to be serialized.
 - f) Use the `encode` method on the object.
 - g) Display the result in the console.
4. You have created a program in Node.js. You want to send an email from your application.
- a) Install `Nodemailer` in your system.
 - b) Import the `Nodemailer`.
 - c) Create an email transport.
 - d) Define the email content subject as `Node.js Working` and add body text as `email sent successfully`.
 - e) Send the email.
5. You are creating Web application using Node.js and you want to implement file upload feature using Node.js.
- a) Install `Formidable` module in your system.
 - b) Import the `Formidable` module.
 - c) Create a form to upload a file.
 - d) Set the response header and write the HTML form.
 - e) Parse the uploaded file.
 - f) Display the message, `File uploaded successfully`.